Dynamic Memory Allocation: Basic concepts

Arquitectura de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

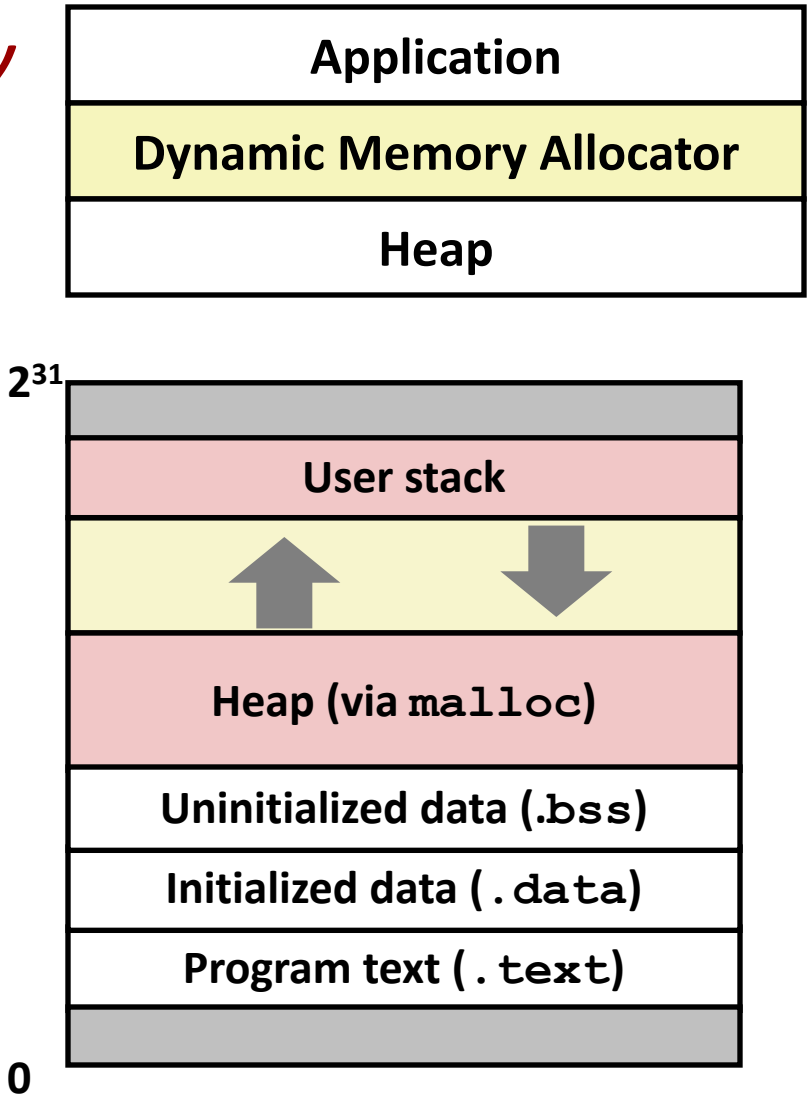
Today

- **Basic concepts**
- **Performance metrics**
- **Implementing an allocator**
  - Implicit free lists

Dynamic memory allocation

- **Programmers use *dynamic memory allocators* (such as `malloc`) to acquire memory at run time**
  - For data structures whose size is only known at runtime

- **Dynamic memory allocators manage an area of process virtual memory known as the *heap***



Dynamic memory allocation

- **Allocator maintains heap as a collection of variable sized *blocks*, which are either *allocated* or *free***

- **Types of allocators**
  - ***Explicit allocator*:** application allocates and frees space
    - E.g., `malloc` and `free` in C
  - ***Implicit allocator:*** application allocates, but does not free space
    - E.g. garbage collection in Java, ML, and Lisp

- **We will discuss simple explicit memory allocation**

The `malloc` package

```
#include <stdlib.h>
```

`void *malloc(size_t size)`

- Successful: returns a pointer to a memory block of at least **`size`** bytes (typically) aligned to 8-byte boundary
  - If **`size == 0`**, returns NULL
- Unsuccessful: returns NULL (0) and sets **`errno`**

`void free(void *p)`

- Deallocates the memory block pointed by **`p`**
- **`p`** must come from a previous call to **`malloc`** or **`realloc`**
- If **`p`** is NULL no operation is performed

The `malloc` package

```
void *calloc(size_t count, size_t size)
```

- Version of **`malloc`** that initializes allocated block to zero

```
void *realloc(void *p, size_t size)
```

- Changes the size of a previously allocated block pointed by **`p`**

malloc example

```
void foo(int n, int m) {
    int i, *p;

    /* Allocate a block of n ints */
    p = (int *) malloc(n * sizeof(int));
    if (p == NULL) {
        perror("malloc");
        exit(0);
    }

    /* Initialize allocated block */
    for (i=0; i<n; i++)
        p[i] = i;


    /* Deallocate p */
    free(p);
}
```

realloc example

```
void foo(int n, int m) {
    int i, new_size, *p;

    /* Allocate a block of n ints */
    p = (int *) malloc(n * sizeof(int));

    …
    scanf("%d",&new_size);

    /* Reallocate the block to its new size */
    p = (int *) realloc(p, new_size * sizeof(int));
    …

    /* Deallocate p */
    free(p);
}
```

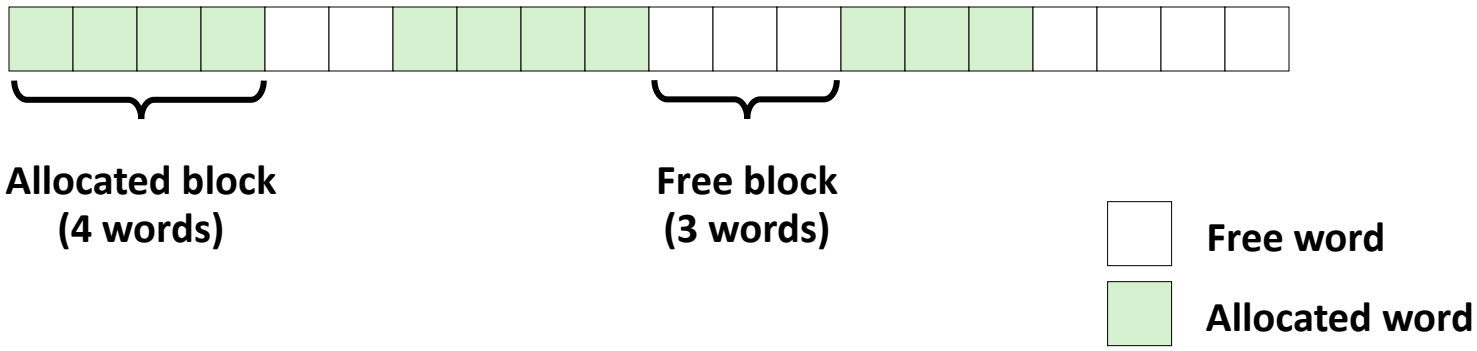
Today

- Basic concepts
- **Performance metrics**
- Implementing an allocator
  - Implicit free lists

Assumptions made in this lecture

- **Memory is word addressed (words are int-sized)**
  - E.g. `malloc(4)` will be equivalent to `malloc(4*sizeof(int))`

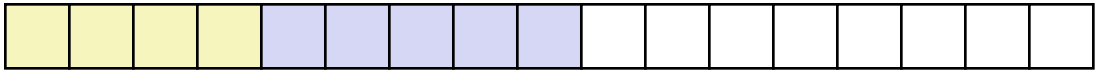
- **Size of allocated blocks is not aligned to multiple of 8 bytes**



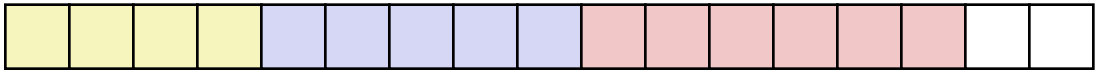
Allocating and freeing blocks

```
p1 = malloc(4)
```

```
p2 = malloc(5)
```



```
p3 = malloc(6)
```



```
free(p2)
```

```
p4 = malloc(2)
```

Dynamic memory allocators

- **Dynamic memory allocation is a useful and important programming technique. However, …**
- **… to use allocators correctly and efficiently, programmers need to have an understanding of how they work**
  - Naive programmers often incorrectly assume that virtual memory is an unlimited resource
- **How can we build a dynamic memory allocator?**
  - Explicit allocators must operate within some rather stringent constraints

Allocator's constraints

1. **Can't control number or size of allocated blocks**
2. **Must respond immediately to `malloc` requests**
   - *i.e.*, can't reorder or buffer requests
3. **Must allocate blocks from free memory**
   - *i.e.*, can only place allocated blocks in free memory
4. **Must align blocks so they satisfy all alignment requirements**
   - 8 byte alignment for GNU `malloc` (`libc malloc`) on Linux
5. **Can manipulate and modify only free memory**
6. **Can't move the allocated blocks once they are `malloc`'d**
   - *i.e.*, defragmentation is not allowed

Performance goals

- **Given some sequence of `malloc` and `free` requests:**
  - $R_0, R_1, \ldots, R_k, \ldots, R_{n-1}$
- **Goals: maximize throughput and peak memory utilization**
- **One of the interesting challenges in any allocator design is finding an appropriate balance between the two goals**
  - As we will see, there is a tension between maximizing throughput and utilization
  - In particular, it is easy to write an allocator that maximizes throughput at the expense of heap utilization

Throughput

- **Number of completed requests per unit time**

- **Example:**
  - 5000 **`malloc`** calls and 5000 **`free`** calls in 10 seconds
  - Throughput is 1000 operations/second

- **In general, we can maximize throughput by minimizing the average time to satisfy allocate and free requests**

Peak memory utilization

- ***Def:* Aggregate payload P_k**
  - `malloc(p)` results in a block with a ***payload*** of `p` bytes
  - After request R_k has completed, the ***aggregate payload*** P_k is the sum of currently allocated payloads

- ***Def:* Current heap size H_k**
  - Let H_k be the high-water mark over the first k requests

- ***Def:* Peak memory utilization after k requests**
  - $U_k = (max_{i<k} P_i) \ / \ H_k$

- **The goal is to maximize the peak utilization over the entire sequence of requests**

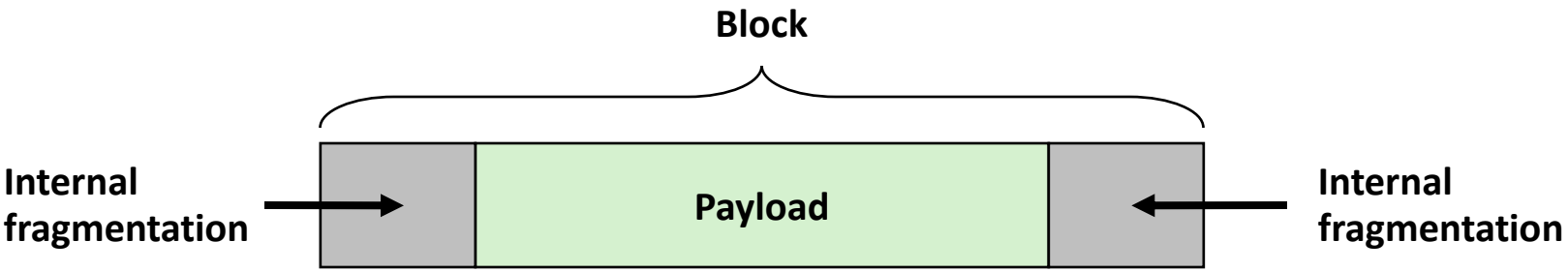
Fragmentation

- **The primary cause of poor heap utilization is a phenomenon known as *fragmentation***
  - Occurs when otherwise unused memory is not available to satisfy allocate requests

- **Two forms of fragmentation**
  - ***Internal*** fragmentation
  - ***External*** fragmentation

Internal fragmentation

- For a given block, *internal fragmentation* occurs if payload is smaller than block size



Internal fragmentation

- **Caused by**
  - Overhead of maintaining heap data structures
  - Padding for alignment purposes
  - Explicit policy decisions (e.g., to return a big block to satisfy a small request)

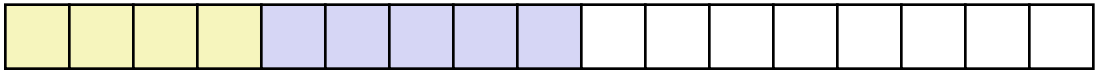
- **Depends only on the pattern of previous requests and the allocator implementation**
  - Thus, easy to measure
  - It is simply the sum of the differences between the sizes of the allocated blocks and their payloads

External fragmentation

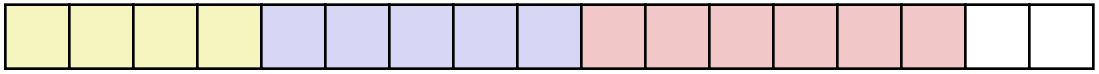
- **Occurs when there is enough aggregate heap memory, but no single free block is large enough**

```
p1 = malloc(4)
```

```
p2 = malloc(5)
```



```
p3 = malloc(6)
```



```
free(p2)
```

```
p4 = malloc(6)
```

Oops! (what would happen now?)

- **Depends also on the pattern of future requests**
  - Thus, difficult to measure

Today

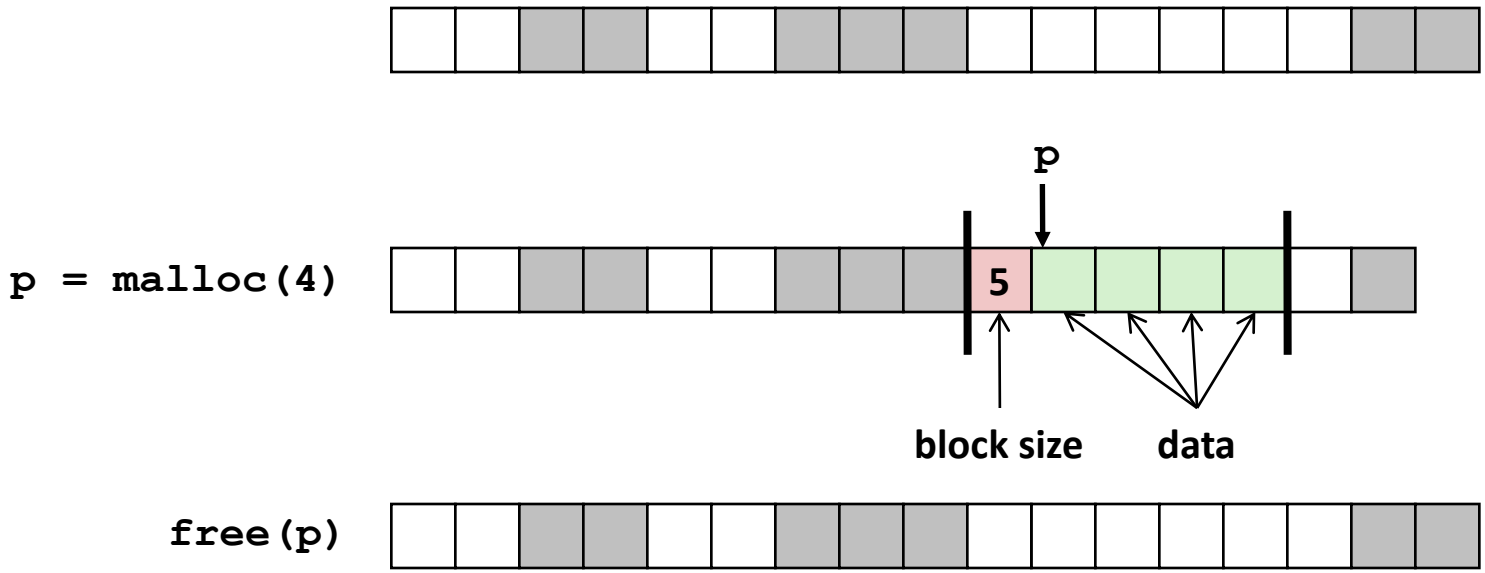
- Basic concepts
- Performance metrics
- **Implementing an allocator**
  - Implicit free lists

Implementation issues: 5 questions

1. How do we know how much memory to free given just a pointer?
2. How do we keep track of the free blocks?
3. How do we pick a block to use for allocation? Many may fit…
4. What do we do with the extra space when allocating a structure that is smaller than the free block it is placed in?
5. How do we reinsert freed block?

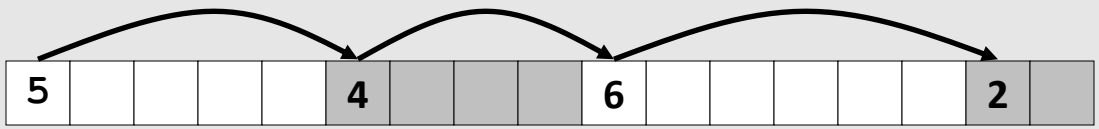
Q1: Knowing how much to free

- **Standard method**
  - Keep the length of a block in the word preceding the block
    - This word is often called the ***header field*** or ***header***
  - Requires an extra word for every allocated block

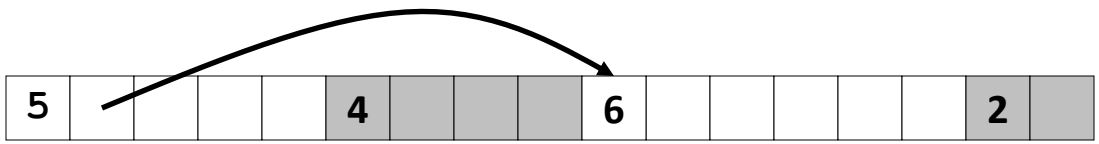


Q2: Keeping track of free blocks

- **Method 1: *Implicit free list* using length—links all blocks**



- **Method 2: *Explicit list* among the free blocks using pointers**



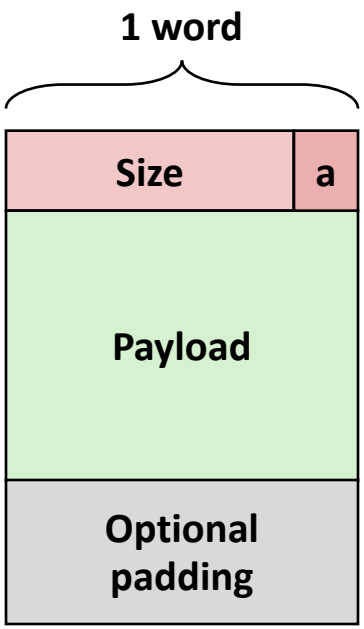
- **Method 3: *Segregated free list***
  - Different free lists for different size classes

- **Method 4: *Blocks sorted by size***
  - Can use a balanced tree (e.g. Red-Black tree) with pointers within each free block, and the length used as a key

Implicit free list

- **For each block we need both size and allocation status**
  - Could store this information in two words: wasteful!
- **Standard method**
  - If blocks are aligned, some low-order address bits are always 0
  - Instead of storing an always-0 bit, use it as a allocated/free flag
  - When reading size word, must mask out this bit

Format of allocated and free blocks

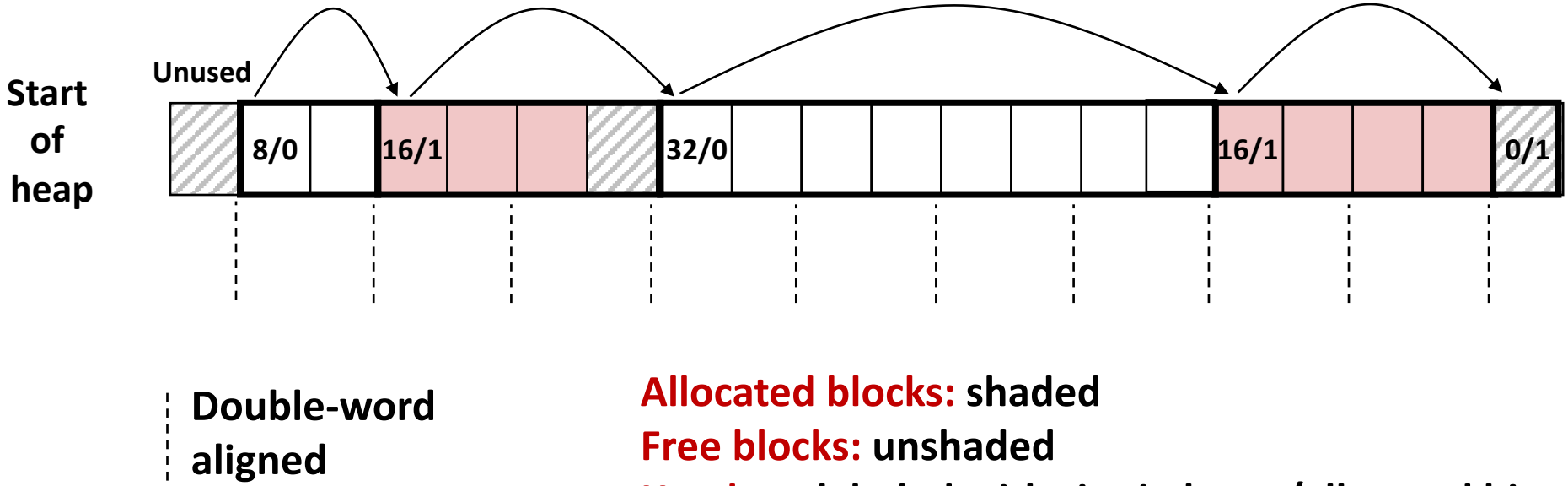


a = 1: Allocated block
a = 0: Free block

Size: block size

Payload: application data (allocated blocks only)

Implicit free list: Example



- **Observations**
  - We need some kind of specially marked end block
  - The system's alignment requirement and the allocator's choice of block format impose a *minimum block size*

Q3: Finding a free block

- **The manner in which the allocator performs this search is determined by the *placement policy***
  - Some common policies are *first fit*, *next fit*, and *best fit*

- **First fit**
  - Search list from beginning, choose first free block that fits
  - Can take linear time in total number of blocks (allocated and free)
  - In practice it can cause "splinters" at beginning of list

Q3: Finding a free block

- **Next fit**
  - Like first fit, but search list starting where previous search finished
  - Should often be faster than first fit: avoids re-scanning unhelpful blocks
  - Some research suggests that fragmentation is worse

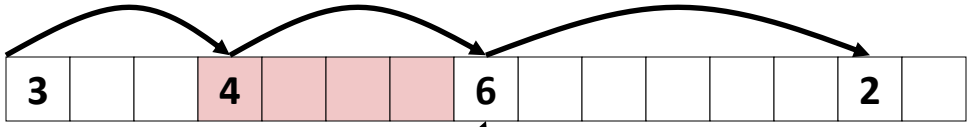
- **Best fit**
  - Search the list, choose the best free block: with fewest bytes left over
  - Trades off lower throughput for less fragmentation

First fit – Example

```
p = start;
while ((p < end) &&       \\ not passed end
       ((*p & 1) ||       \\ already allocated
       (*p  <= len)))     \\ too small
  p = p + (*p & -2);      \\ goto next block
                          \\ (word addressed)
```

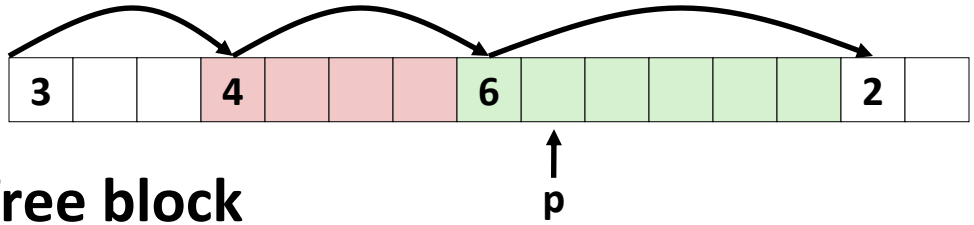
Q4: Allocating in free block

Suppose we need to allocate 3 words

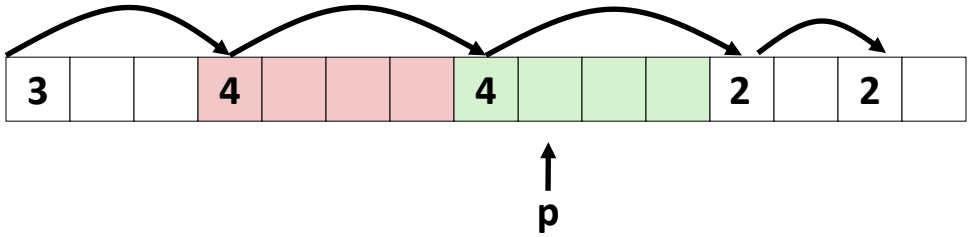


This is our free block of choice

1. **Allocate the whole block → internal fragmentation!**



2. **Split the free block**

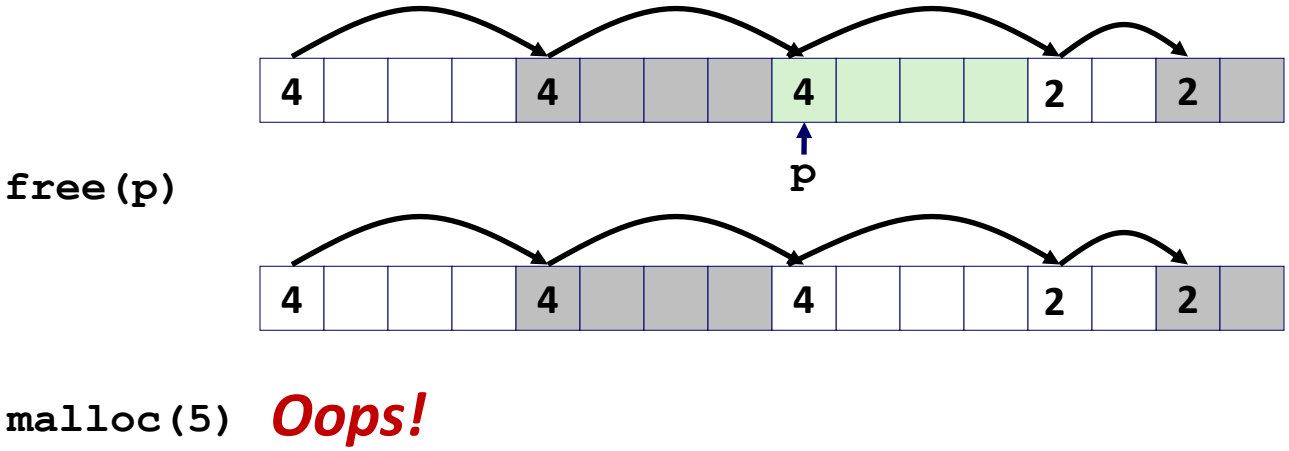


Split the free block – Example

```
void addblock(ptr p, int len) {
  // round up to even
  int newsize = ((len + 1) >> 1) << 1;

  // mask out low bit
  int oldsize = *p & -2;

  // set new length
  *p = newsize | 1;

  // set length in remaining part of block
  if (newsize < oldsize)
    *(p+newsize) = oldsize - newsize;
}
```

Q5: Freeing a block

- **Simplest implementation: clear the "allocated" flag**
  - Can lead to "false fragmentation"

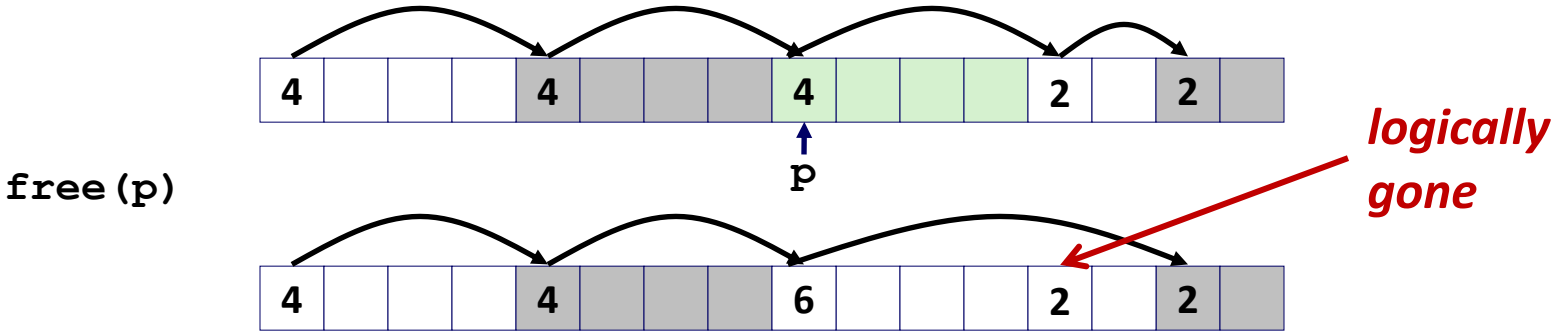


There is enough free space, but the allocator won't be able to find it

Implicit list: Aggregating

- Aggregate with next/previous blocks, if they are free

- Joining with next block



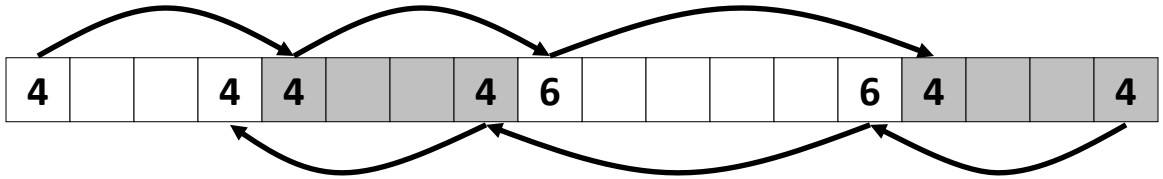
Aggregation – Example

```
void free_block(ptr p) {
    *p = *p & -2;              // clear allocated flag
    next = p + *p;             // find next block
    if ((*next & 1) == 0)
      *p = *p + *next;         // add to this block if
}                              //    not allocated
```

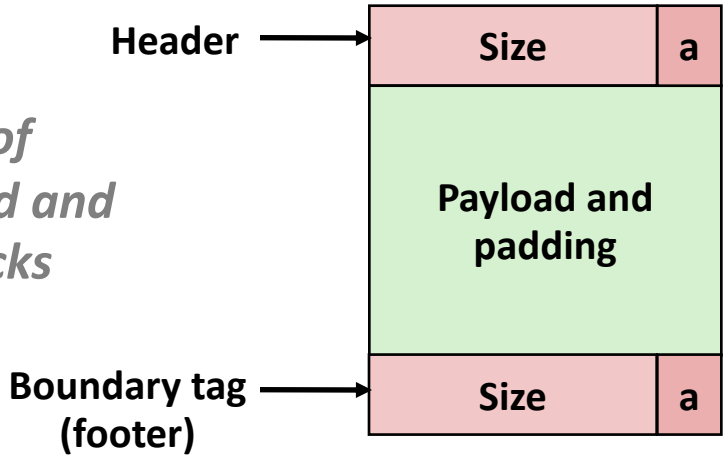
- **But how do we aggregate with *previous* block?**

Implicit list: Bidirectional aggregation

- ***Boundary tags*** [Knuth73]
  - Replicate size/allocated word at "bottom" (end) of free blocks
  - Allows us to traverse the "list" backwards, but requires extra space



Format of allocated and free blocks



a = 1: Allocated block
a = 0: Free block

Size: Total block size

Payload: Application data (allocated blocks only)

Summary: Implicit lists

- **Implementation: very simple**
- **Allocation cost: linear-time in worst-case**
- **Freeing cost: constant-time in worst-case, even with aggregation**
- **Memory usage: will depend on placement policy**
- **Application: Not used in practice for `malloc/free` because of linear-time allocation**
  - However, the concepts of splitting and boundary tag aggrgation are general to *all* allocators

More info on allocators

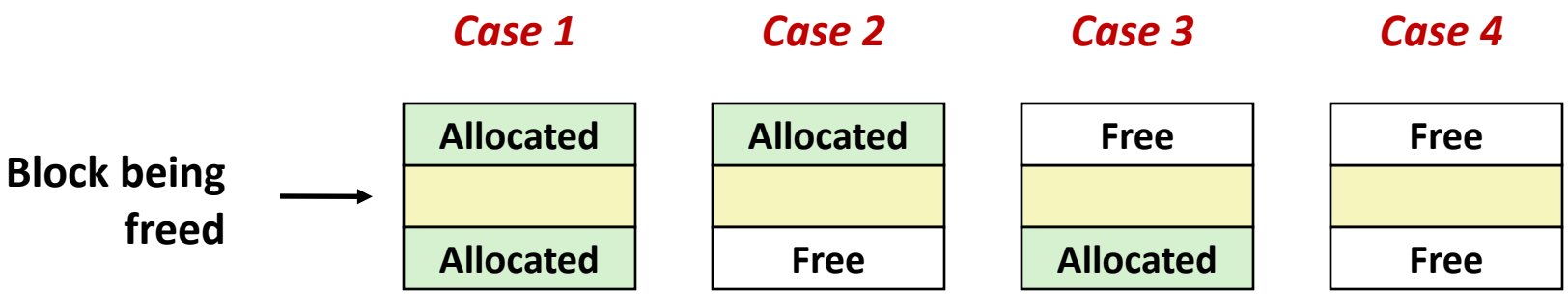
- **D. Knuth, "*The Art of Computer Programming*", 2nd edition, Addison Wesley, 1973**
  - The classic reference on dynamic storage allocation

- **Wilson et al, "*Dynamic Storage Allocation: A Survey and Critical Review*", Proceedings of the 1995 International Workshop on Memory Management, Kinross, Scotland, Sept, 1995**
  - Comprehensive survey

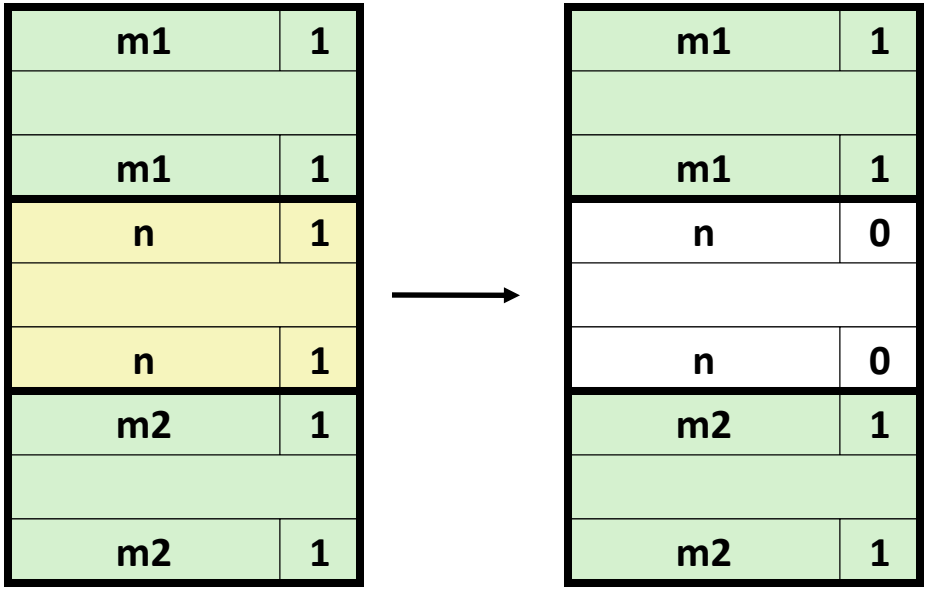
Recommended reading

SUPPLEMENTAL SLIDES

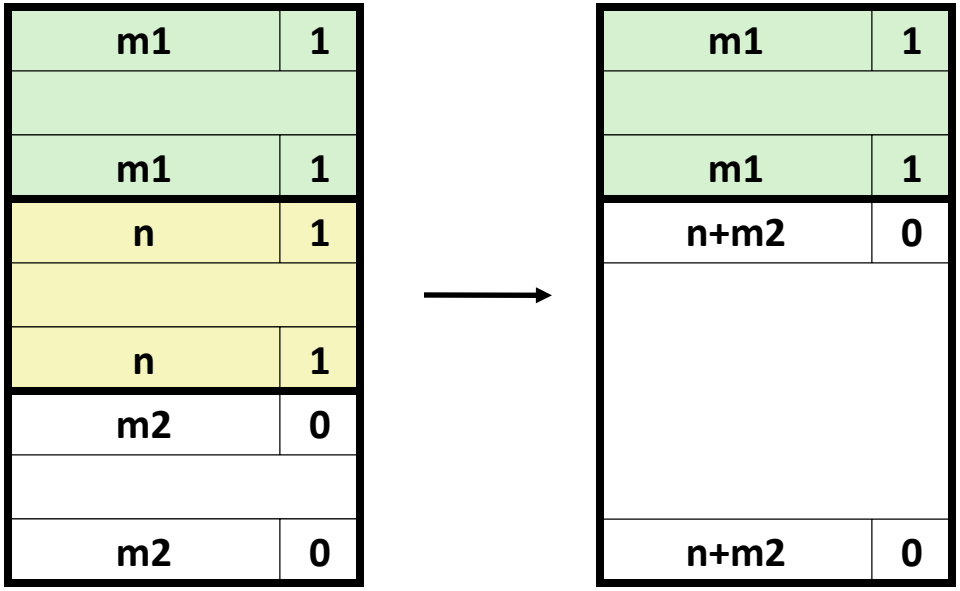
Constant time aggregation



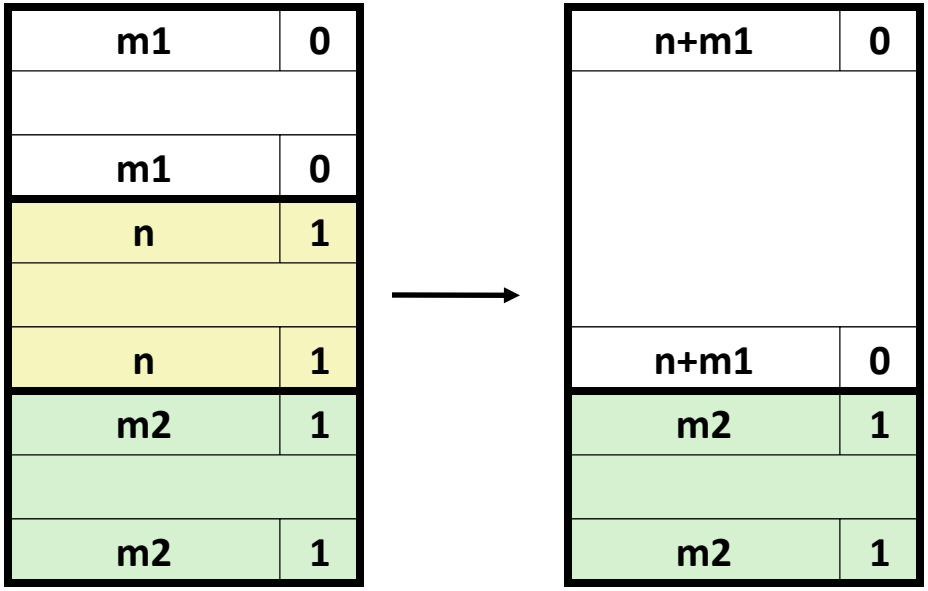
Constant time aggregation (Case 1)



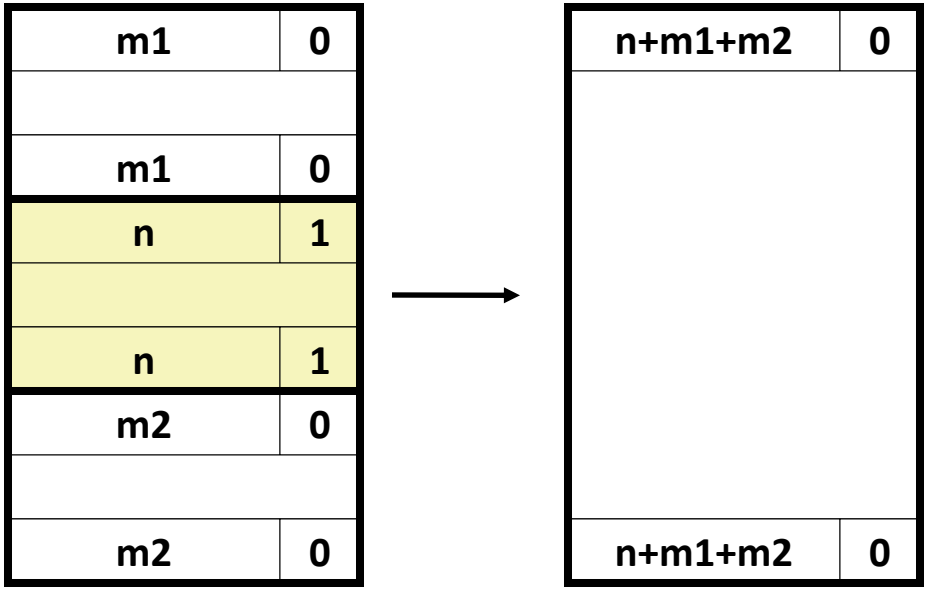
Constant time aggregation (Case 2)



Constant time aggregation (Case 3)



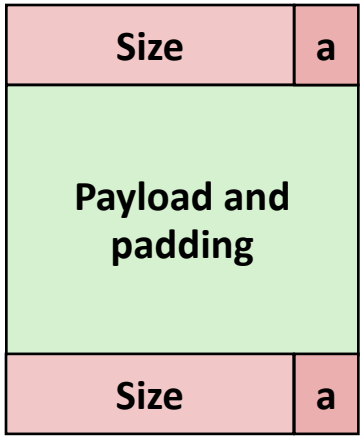
Constant time aggregation (Case 4)



Disadvantages of boundary tags

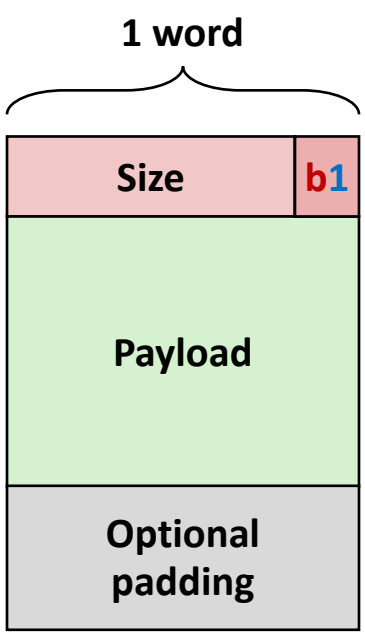
- **Internal fragmentation**

- **Can it be optimized?**
  - Which blocks need the footer tag?
  - What does that mean?



No boundary tag for allocated blocks

- **Boundary tag needed only for free blocks**
  - When sizes are multiples of 4 or more, have 2+ spare bits

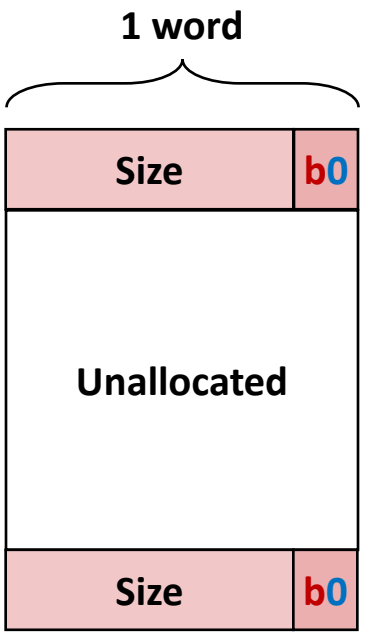


Allocated Block

a = 1: Allocated block
a = 0: Free block
b = 1: Previous block is allocated
b = 0: Previous block is free

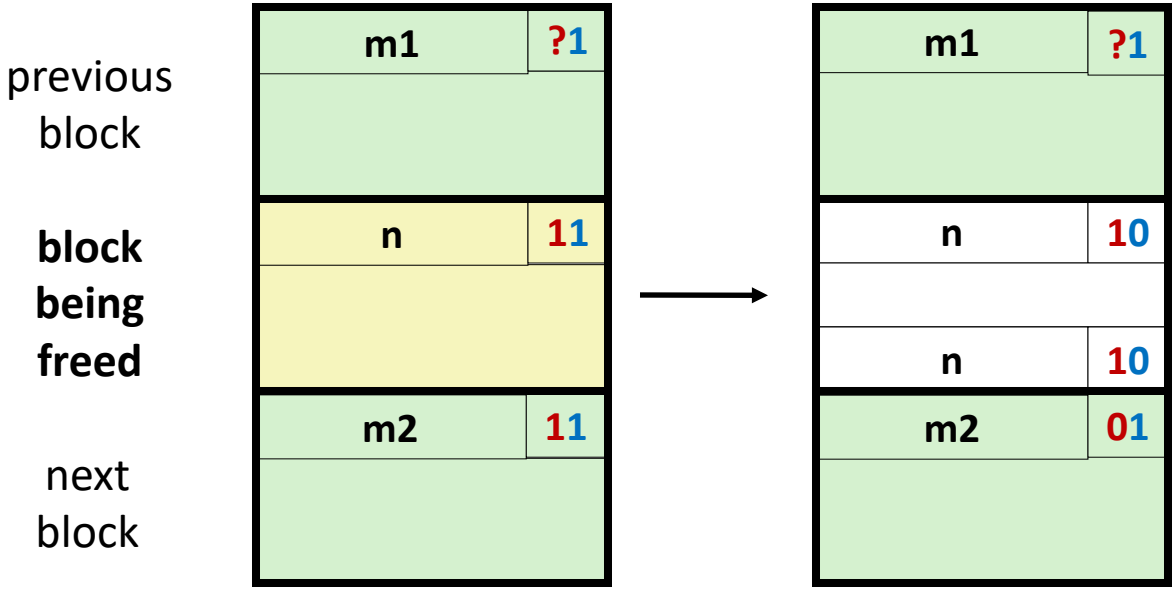
Size: block size

Payload: application data



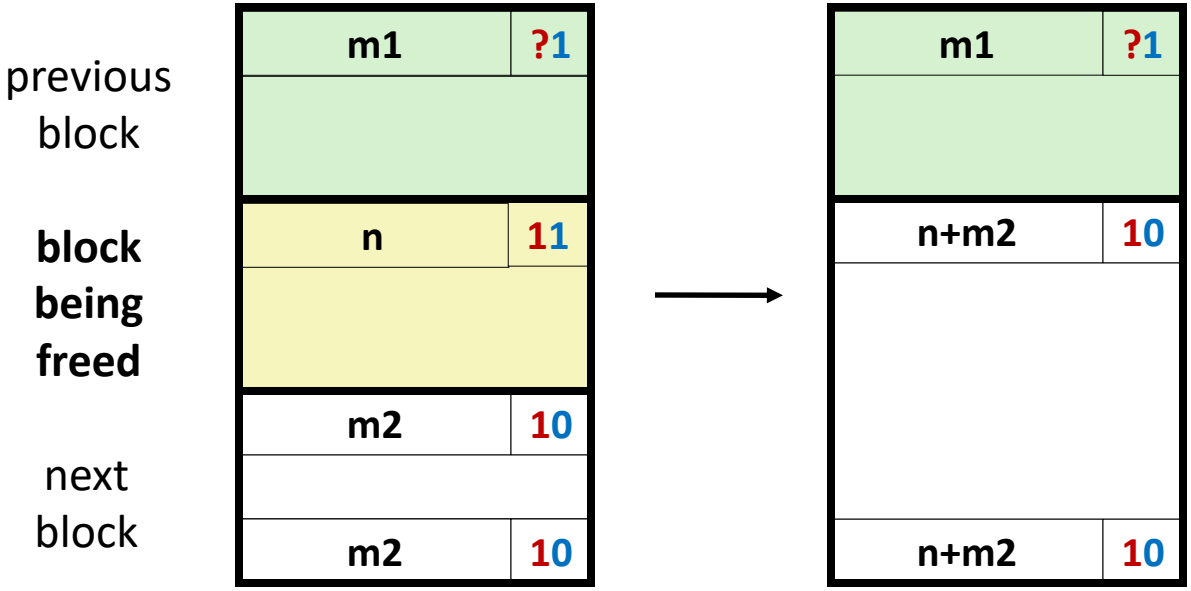
Free Block

No boundary tag for allocated blocks (Case 1)



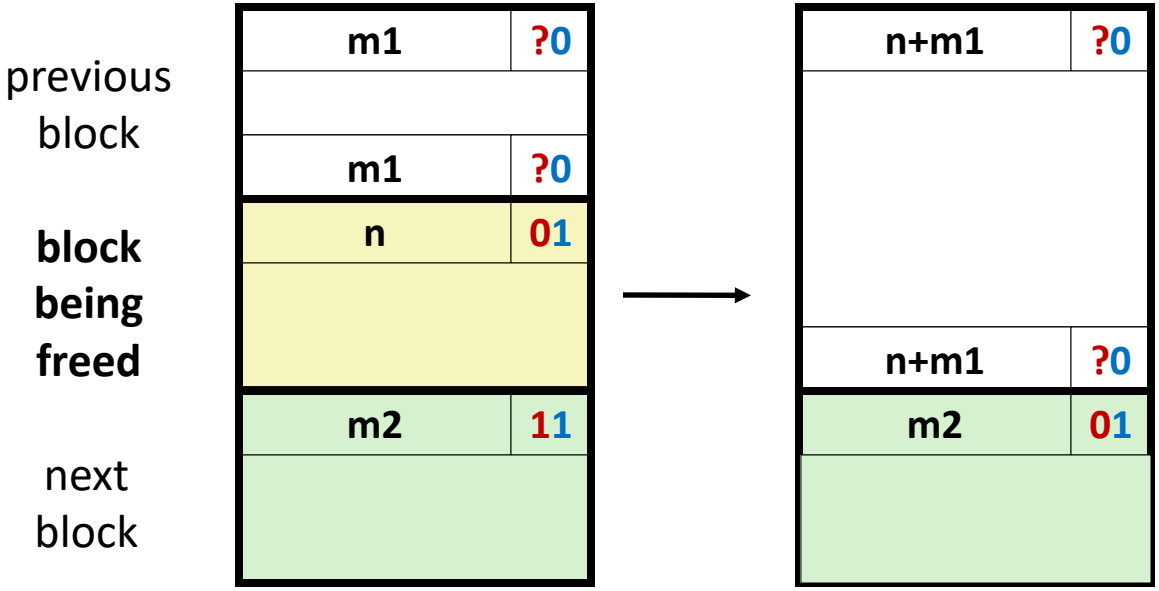
Header: Use 2 bits (address bits always zero due to alignment):
(previous block allocated)<<1 | (current block allocated)

No boundary tag for allocated blocks (Case 2)



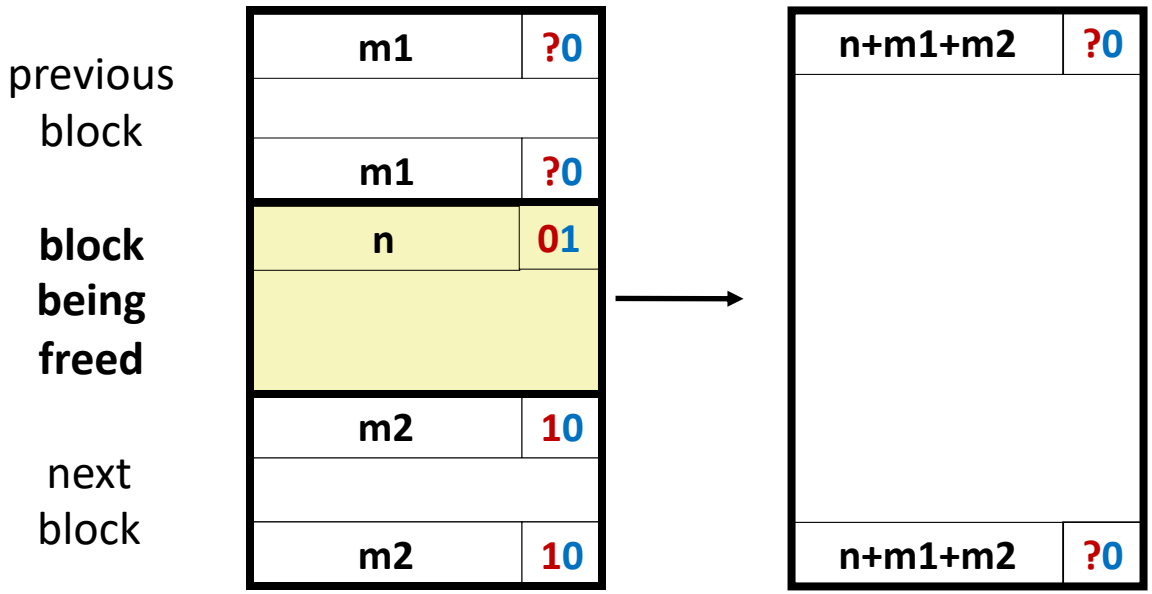
Header: **Use 2 bits (address bits always zero due to alignment):**
(previous block allocated)<<1 | (current block allocated)

No boundary tag for allocated blocks (Case 3)



Header: **Use 2 bits (address bits always zero due to alignment):**
(previous block allocated)<<1 | (current block allocated)

No boundary tag for allocated blocks (Case 4)



Header: **Use 2 bits (address bits always zero due to alignment):**
(previous block allocated)<<1 | (current block allocated)